

ST-BLR Interface and Output Map

Peter Sørensen

Table of contents

1	Main Modelling Interfaces	1
2	Shared Data Object	2
3	CSR Workflow	2
4	BED Workflow	3
5	Model Family Map	3
6	selection_s Support	6
7	Standardized Output Fields	7
8	Helper Function Map	7
9	Fine-Mapping, LD-Swap, and Credible Sets	9

This note is a compact map of the current ST-BLR user-facing interfaces, shared data object, model families, standardized outputs, and post-processing helpers. It is intentionally not a replacement for function help pages.

1 Main Modelling Interfaces

- `stblr_csr()`: summary-statistics CSR backend using `stats` and sparse LD from `Glist` or an explicit `ld_prefix`.
- `stblr_bed()`: individual-level PLINK/BED backend using genotype information from `Glist`.
- `stblr_csr_annot()`: unified annotation-aware CSR backend using `annotation_model = "prior", "learned", "group", or "sbayesrc"`.

`stblr_bed_marker()` remains available as a lower-level direct BED marker interface, but the high-level BED model entry point is `stblr_bed()`.

2 Shared Data Object

`Glist` is the common carrier for genotype and marker metadata used by both CSR and BED workflows. In current code, relevant fields include:

- `Glist$bedfiles`, the PLINK BED file paths used by BED and preparation helpers;
- `Glist$n`, the number of BED individuals;
- `Glist$ids` and optionally `Glist$idsLD`, used for sample matching;
- `Glist$rsids`, marker IDs in BED-file order by chromosome/file;
- `Glist$rsidsLD`, selected LD/model marker IDs by chromosome/file;
- `Glist$af`, allele frequencies used for marker standardization and MAF dependent calculations;
- `Glist$sparseLD`, added by `make_sparseLD()`, with `prefix`, `chr`, `bed_files`, `cls`, `af`, `rows`, `max_distance_bp`, `max_distance_variants`, `r2_threshold`, and `block_size`.

For sparse-LD CSR models, marker-level information such as MAF must be aligned to the LD marker order. The package preparation helpers use positional alignment equivalent to:

```
idx <- match(Glist$rsidsLD[[chr]], Glist$rsids[[chr]])
```

Do not use `%in%` as the recommended alignment operation because it loses the requested LD marker order.

3 CSR Workflow

1. Prepare or load a BED-backed `Glist`.
2. Compute summary statistics with `make_stats()`.
3. Compute sparse LD with `make_sparseLD()`.
4. Fit summary-statistics BLR models with `stblr_csr()` or annotation-aware CSR models with `stblr_csr_annot()`.

```
stats <- make_stats(  
  Glist,  
  y,  
  nthreads = nthreads  
)
```

```
Glist <- make_sparseLD(  
  Glist = Glist,  
  out_prefix = file.path(data_dir, "ld_test"),  
  pos_bp = NULL,  
  max_distance_bp = 0,  
  max_distance_variants = 1000,  
  r2_threshold = 0.001,  
  block_size = 1024,  
  nthreads = 4
```

```
)

fit_csr <- stblr_csr(
  stats = stats,
  Glist = Glist,
  method = "bayesC"
)
```

`make_stats()` computes marker-trait summary statistics used by `stblr_csr()`. `make_sparseLD()` computes disk-backed sparse LD and stores sparse-LD metadata in `Glist$sparseLD`. The updated `Glist` is then passed to `stblr_csr()` and related CSR functions.

4 BED Workflow

1. Prepare or load a BED-backed `Glist`.
2. Fit individual-level BLR models using `stblr_bed()`.
3. Use standardized output fields where available.

```
fit_bed <- stblr_bed(
  Glist = Glist,
  y = y,
  method = "bayesC"
)
```

`stblr_bed()` supports `method = "bayesC"` and `method = "bayesR"`. Its current arguments include phenotype data `y`, `Glist`, optional `chr`, `cls`, `rows`, `scale`, MCMC controls, BayesC prior controls, BayesR mixture controls, and BED scheduled-chain controls. `covar` is not currently adjusted by this interface; pass pre-adjusted phenotypes. `chain_seeds` are not currently supported by the BED scheduled-chain backends.

5 Model Family Map

Family	Function	Backend	Requires sparse LD	Uses in- dividual BED geno- types	Uses annota- tions	Component proba- bilities	selections	LD- swap / fine- mapping	Main out- puts
Annotation- unaware	tblr_csr_bayesc method or CSR = sched- BayesC "bayesC" CSR BayesC		yes	no	no	no	fixed and sampled in unsched- uled CSR; not sched- uled	LD- swap for unsched- uled CSR with updateLDswap = TRUE; fine- mapping helpers use fit\$dm and LD	dm, bm, vbs, vgs, ves, vle, swap, input, op- tional chains
Annotation- unaware	tblr_csr_bayesr method CSR = BayesR "bayesR")		yes	no	no	yes, component_0 is null	fixed and sampled	LD- swap with updateLDswap = TRUE; fine- mapping helpers use fit\$dm and LD	common out- puts vars comp_prob, dm_component_mean, ncomp where avail- able
Individual- level	tblr_bed.. method BED = chain BayesC "bayesC" back- end		no	yes	no	no	no	LD- swap; compati- ble with post- processing that only needs fit\$dm/fit\$bm matrices	dm, bm, input, CPO/log- CPO diag- nos- tics, chain- summary matrices

Family	Function	Backend	Requires sparse LD	Uses in- dividual BED geno- types	Uses annota- tions	Component proba- bilities	selections	LD- swap / fine- mapping	Main out- puts
Individual level BED BayesR	<code>stblr_bed</code> method = <code>"bayesR"</code>	BED., BayesR scheduled- chain back- end	no	yes	no	yes, <code>component_0</code> is null	no	no LD- swap; compati- ble with post- processing that only needs <code>fit\$dm/fit\$bm</code>	BED com- mon out- puts plus <code>comp_prob</code> , <code>dm_component_mean</code>
Fixed- prior anno- tation CSR	<code>stblr_csr</code> annotation_model = <code>"prior"</code>); wrap- per <code>stblr_csr_prior_annot()</code>	<code>stblr_csr_annot</code> , BayesR	yes	no	fixed marker priors	no	no	LD- swap where enabled; no sampler- level selections	common out- puts plus <code>annotation_prior/a</code> where re- turned
Learned anno- tation CSR	<code>stblr_csr</code> annotation_model = <code>"learned"</code>); wrap- per <code>stblr_csr_learn_annot()</code>	<code>stblr_csr_annot</code> , BayesR	yes	no	learned inclu- sion/variance effects	no	no	LD- swap where enabled; no sampler- level selections	common out- puts plus <code>eta_pi</code> , <code>eta_vb</code> , <code>annotation_effects</code> <code>annotation_summary</code>
Group anno- tation CSR	<code>stblr_csr</code> annotation_model = <code>"group"</code>); wrap- per <code>stblr_csr_group_annot()</code>	<code>stblr_csr_group</code> , BayesR	yes	no	mutually exclu- sive groups	no	no	LD- swap where enabled; no sampler- level selections	common out- puts plus <code>group_pi</code> , <code>group_vb_multiplie</code> <code>group_nincluded</code> , <code>group_size</code> , <code>annotation_summary</code>

Family	Function	Backend	Requires sparse LD	Uses in- dividual BED geno- types	Uses annota- tions	Component proba- bilities	selection_s	LD- swap / fine- mapping	Main out- puts
SBayesRC	stblr_csr_sbayesc;	yes		no	component	yes, gamma_0.00 is null	fixed and sampled	LD- swap sup- ported	common out- puts plus comp_prob, dm_component_mean, alpha, sigmaSqAlpha, ncomp
style	annotation_model				proba- bilities				
anno-	=								
tation	"sbayescr");								
CSR	wrap- per stblr_csr_sbayesc_generic()								

6 selection_s Support

Fixed global `selection_s` and sampled trait-specific `selection_s` are supported for:

- `csr_bayesc;`
- `csr_bayescr;`
- `csr_sbayescr.`

They are not supported for:

- `csr_scheduled_bayesc;`
- `csr_prior_bayesc;`
- `csr_annot_bayesc;`
- `csr_group_bayesc;`
- BED backends.

Sampled-S defaults are:

```
selection_s_init = 0
selection_s_prior = c(-3, 2)
selection_s_proposal_sd = 0.35
```

CSR effects are on the standardized-genotype scale. The MAF-dependent prior scale is:

$$q_j(S) = h_j \cdot (S + 1)$$

$$h_j = 2 \cdot p_j \cdot (1 - p_j)$$

7 Standardized Output Fields

Model outputs are standardized across backends where possible:

- `fit$dm`: marker-by-trait posterior inclusion probability or non-null probability.
- `fit$bm`: marker-by-trait posterior mean effect.
- `fit$vb`: posterior trace of marker-effect variance.
- `fit$vg`: posterior trace of genetic variance.
- `fit$ve`: posterior trace of residual variance.
- `fit$vl`: posterior trace of linkage-equilibrium contribution, where returned.
- `fit$vd`: posterior trace of LD contribution, where returned.
- `fit$input`: backend, model, settings, and metadata.
- `fit$chains`: optional compact chain-level output when supported and `keep_chains = TRUE`.

BayesR and SBayesRC outputs:

- `fit$comp_prob`: marker-by-component posterior probabilities by trait.
- `fit$dm_component_mean`: posterior mean component/effect-class summary.
- BayesR null component: `component_0`.
- SBayesRC null component: `gamma_0.00`.

Annotation-aware outputs are model-specific and may include:

- `fit$annotation_summary`;
- `fit$annotation_pi`;
- `fit$annotation_effects`;
- `fit$annotation_prior`;
- `fit$eta_pi`, `fit$eta_vb` for learned annotation BayesC;
- `fit$group_pi`, `fit$group_vb_multiplier`, `fit$group_nincluded`, `fit$group_size` for group BayesC;
- `fit$alpha`, `fit$sigmaSqAlpha`, and `fit$ncomp` for SBayesRC-style fits;

Sampled-S outputs:

- `fit$selection_s`;
- `fit$selection_s_sd`;
- `fit$selection_s_min`;
- `fit$selection_s_max`;
- `fit$selection_s_trace`;
- `fit$selection_s_acceptance`.

With `keep_chains = TRUE`, supported CSR chain objects include:

- `fit$chains[[trait]][[chain]]$selection_s`;
- `fit$chains[[trait]][[chain]]$selection_s_acceptance`.

8 Helper Function Map

Helper	Consumes	Returns	Applies to	Notes
<code>summarise_stblr_posterior</code>	As <code>fit\$traces</code> (trace matrices such as <code>vbs</code> , <code>vgs</code> , <code>ves</code> , <code>pis</code> , <code>vle</code> , <code>vld</code>)	Data frame of posterior means, intervals, diagnostics, and derived quantities	Fits with compatible traces	General posterior trace summary helper
<code>plot_stblr_posterior</code>	Output from <code>summarise_stblr_posterior</code>	Base R posterior plot, invisibly returns plotted data	Any compatible posterior summary	Plot helper, not a sampler
<code>summarise_stblr_bayesr_components</code>	BayesR/SBayesRC-style fit with <code>dm</code> and <code>comp_prob</code>	Trait-level component and PIP diagnostics, optionally unstable markers	BayesR/SBayesRC component fits	Uses <code>component_0</code> for BayesR and the detected null component such as <code>gamma_0.00</code> for SBayesRC
<code>summarise_stblr_maf_heterozygosity</code>	Affix architecture plus <code>maf</code> or <code>h</code> , or recoverable MAF/heterozygosity	(Post-hoc MAF architecture regression summary)	Any fit with marker summaries	Descriptive post-hoc diagnostic, distinct from sampler-level <code>selection_s</code>
<code>make_credible_sets</code>	Named PIP vector and dense regional LD matrix	Credible-set summary, sets, aligned PIP, parameters	Any named PIP/LD input	R-level post-processing
<code>make_multisignal_credible_sets</code>	Named PIP vector and dense regional LD matrix	Approximate multi-signal credible-set summary and sets	Any named PIP/LD input	Uses marginal PIPs, not posterior inclusion configurations
<code>make_stblr_credible_sets</code>	Fit sets (<code>fit\$dm</code> , optional <code>Glist</code> , optional dense LD, optional marker sets)	Combined summary, nested sets, loci, parameters, and resolved trait	ST-BLR fits with marker-level PIPs	Uses <code>fit\$dm</code> ; can extract regional LD from <code>Glist\$sparseLD</code>
<code>extract_stblr_fine_map</code>	Genome-wide fit, <code>Glist</code> , and predefined locus sets	<code>stblr_fine_map</code> object with summary, markers, optional credible_sets, loci, parameters	Genome-wide fits with named <code>fit\$dm/fit\$bm</code>	Post-processing only; does not rerun MCMC

Helper	Consumes	Returns	Applies to	Notes
<code>finemap_stblr_csr()</code>	Global CSR fit, <code>Glist</code> , <code>stats</code> , and local marker sets	<code>stblr_finemap</code> object with local run summaries and optional credible sets	CSR fits	Runs local CSR fine-mapping using global architecture estimates
<code>check_stblr_backend_consistency()</code>	Backend ST-BLR fit	Backend consistency check object	Standardized fit objects	Validates expected fields, dimensions, metadata, and optional LD-swap output
<code>check_stblr_convergence()</code>	Filtered MCMC trace matrices	Simple convergence diagnostic list	Fits with compatible traces	Screening diagnostics, not proof of convergence

9 Fine-Mapping, LD-Swap, and Credible Sets

LD-swap diagnostics are available through `fit$ld_swap` and chain-level `ld_swap / ld_swap_chains` output when `updateLDswap = TRUE` and the backend supports it. LD-swap is a sampler mixing diagnostic and proposal mechanism, not a standalone credible-set object.

PIP is represented by `fit$dm`. Credible sets are constructed by helper functions such as `make_stblr_credible_sets()`, `make_credible_sets_from_ld()`, and `make_multisignal_credible_sets_from_ld()` from PIPs and LD. They are workflow/helper-level diagnostics unless a specific fine-mapping helper returns them in its own post-processing object.

`make_stblr_credible_sets()` requires a fit containing marker PIPs in `fit$dm`. It can use a supplied dense LD matrix or, for CSR workflows, use `Glist$sparseLD$prefix` to extract regional dense LD from disk-backed sparse LD. It defines automatic loci around high-PIP markers when explicit `sets` are not supplied. Credible-set probability is cumulative marginal PIP within the selected LD neighborhood, reported as `cs_pip`; the helper returns `summary`, nested `sets`, `locus_sets`, `loci`, `parameters`, and the resolved `trait`.